AWS Multi-Region VPC Peering and Network Setup

Introduction

Cloud environments rarely exist inside a single isolated network. Modern infrastructures are usually distributed across multiple regions, segmented into public and private networks, and designed with strict access control rules.

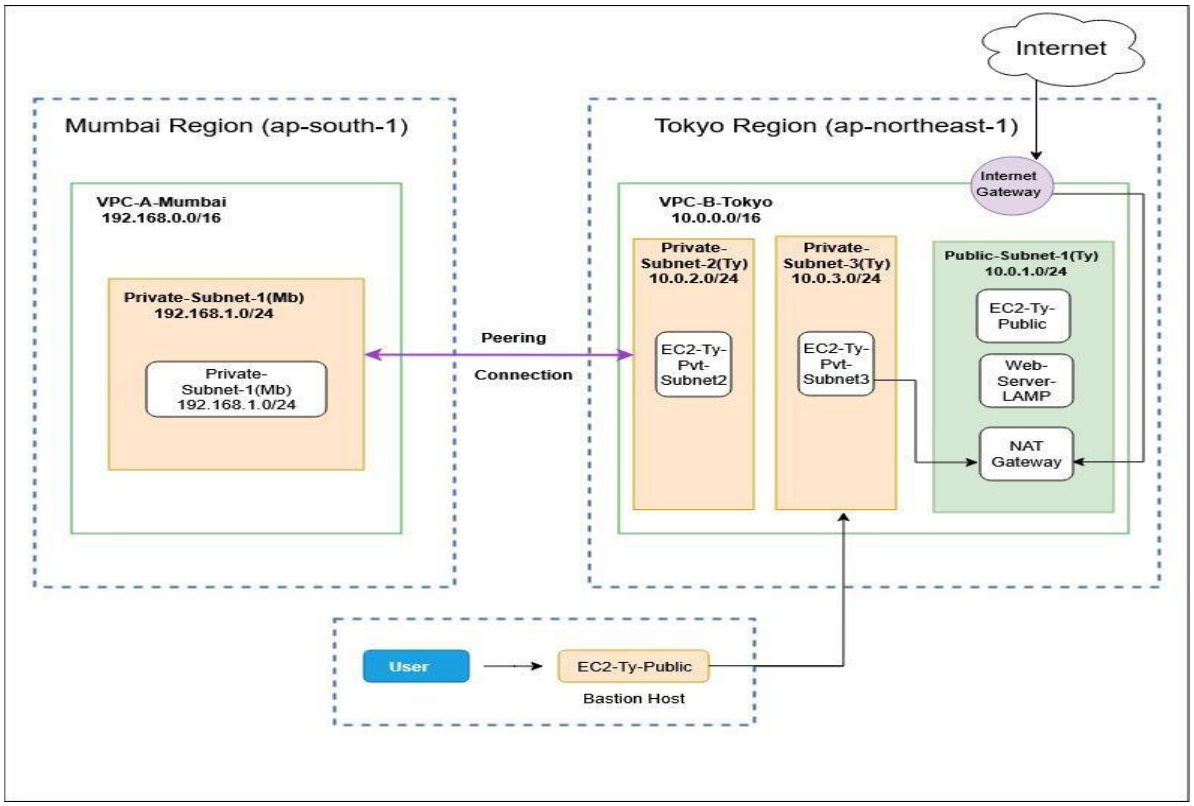
In this project, I built a multi-region AWS networking architecture connecting two VPCs across different regions using VPC Peering, while also implementing secure internet access for private resources using a NAT Gateway. To complete the setup, I deployed a LAMP stack server inside the public subnet.

Project Architecture

The infrastructure spans two AWS regions:

Region	Purpose
Mumbai (ap-south-1)	Private application network
Tokyo (ap-northeast-1)	Public and internal network

Architecture Diagram



This architecture spans two AWS regions and demonstrates how secure communication can be established between isolated cloud networks. The Mumbai VPC contains a private subnet designed for internal communication, while the Tokyo VPC hosts both public and private workloads. Internet access for private resources is managed using a NAT Gateway, and cross-region connectivity is achieved through VPC Peering.

Step 1: Create VPC A in Mumbai Region with Private Subnet

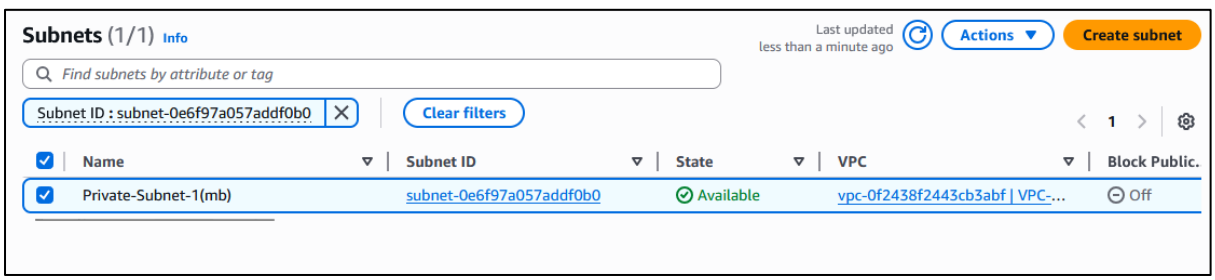
Firstly, switch to region **Mumbai (ap-south-1)** and create the VPC

1. Navigate to **VPC Dashboard → Your VPCs**
2. Fill in the details:
    - **Name:** VPC-A-Mumbai
    - **IPv4 CIDR block**: 192.168.0.0/16
3. Click **Create VPC**



After creating the VPC, create a private subnet inside it.

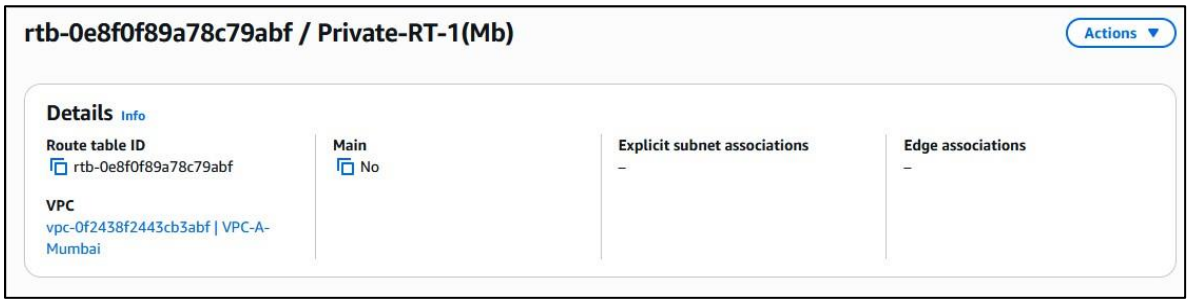
1. In VPC Dashboard, select **Subnets → Create Subnet**
2. Configure:
    - **VPC ID**: VPC-A-Mumbai
    - **Subnet name**: Private-Subnet-1(mb)
    - **Availability Zone**: ap-south-1a
    - **IPv4 CIDR block**: 192.168.1.0/24
3. Click **Create Subnet**



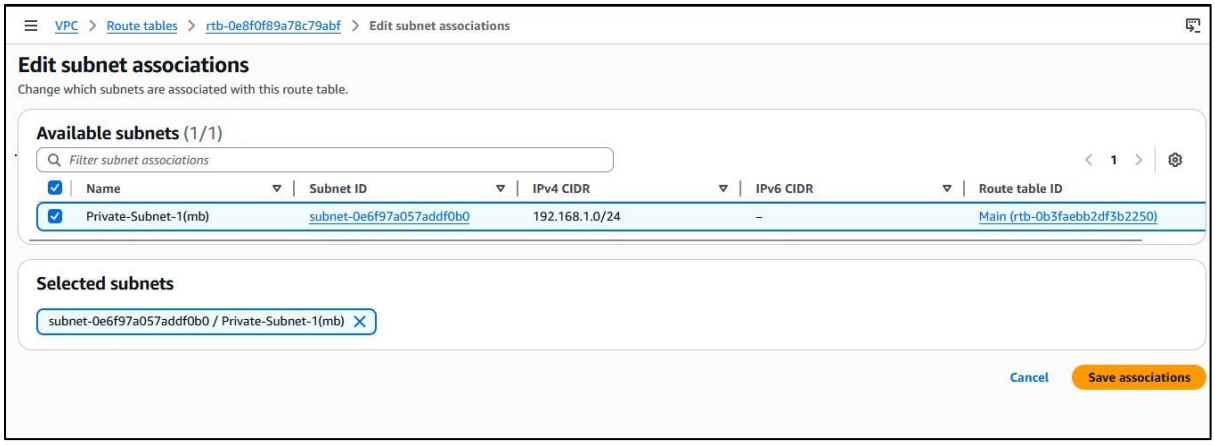
Since this subnet was intended to remain private, auto-assign public IP was disabled.

For every subnet, a route table has to be created.

1. Go to **Route Tables → Create Route Table**
2. **Route Table name:** Private-RT-1(Mb)
3. **VPC:** VPC-A-Mumbai
4. Click **Create**



5. In the route table details, keep the local route (192,168.0.0/16 → Local)
6. Go to **Subnet Association** tab → **Edit Subnet Associations**
    - Check the 'Private-Subnet-1(Mb)'



Note: This subnet has no internet gateway attachment, making it truly private.

Step 2: Create VPC B in Tokyo Region with Three Subnets

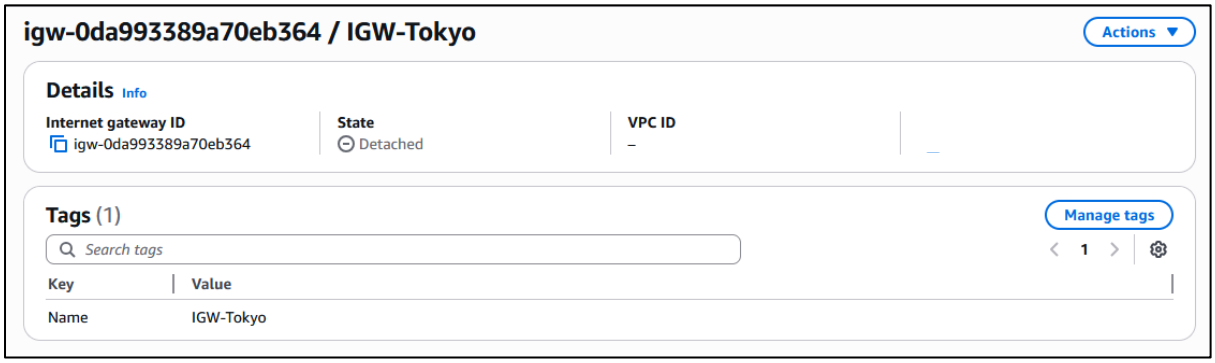
Switch to region **Tokyo (ap-northeast-1)** and create a VPC.

1. Navigate to **VPC Dashboard → Create VPC**
2. Configure:
    - **Name:** VPC-B-Tokyo
    - **IPv4 CIDR block**: 10.0.0.0/16
3. Click **Create VPC**

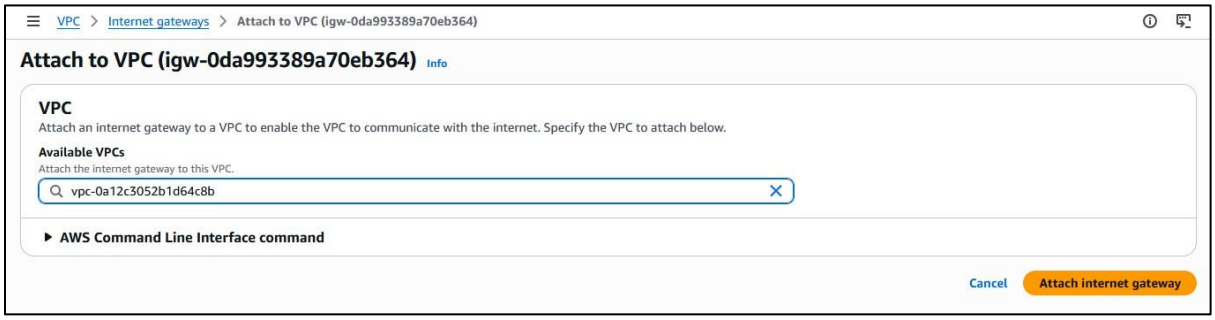


Since the Tokyo VPC would contain public-facing resources, internet connectivity needed to be configured using an Internet Gateway. It is required for public subnet communication with the internet.

1. Go to **Internet Gateways → Create Internet Gateway**
2. **Name:** IGW-Tokyo
3. Click **Create**

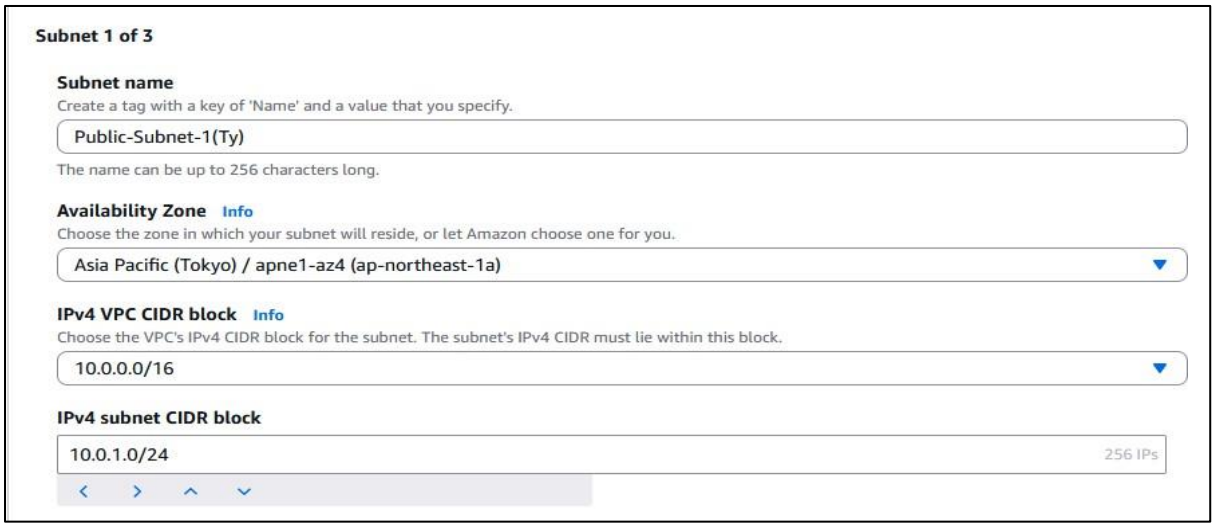


4. Select the created IGW, then **Attach to VPC**
5. Select **VPC-B-Tokyo** and confirm



We have to create three subnets – one public subnet and two private subnets.

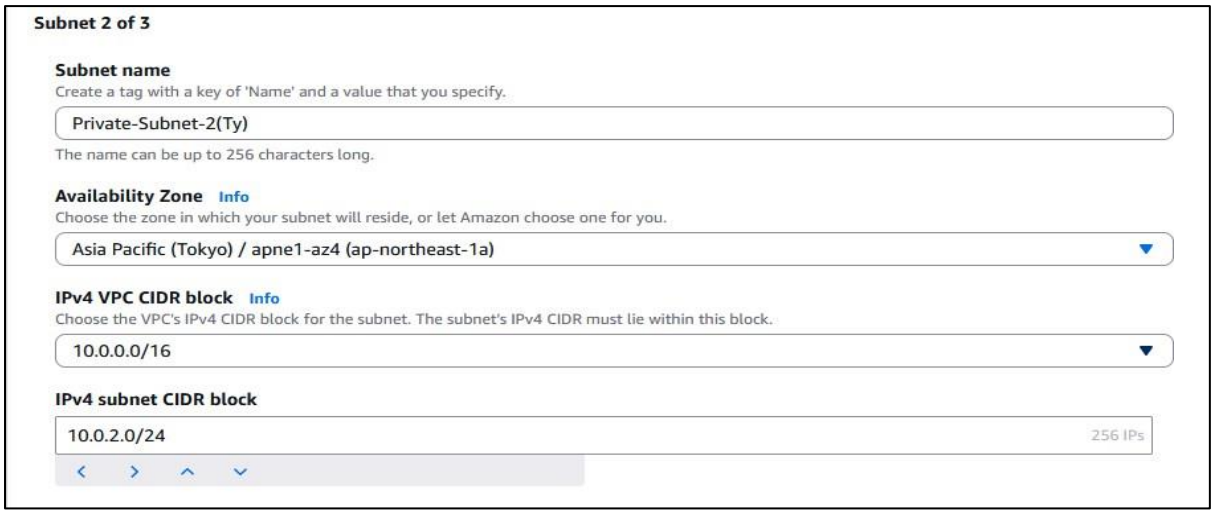
Subnet 1



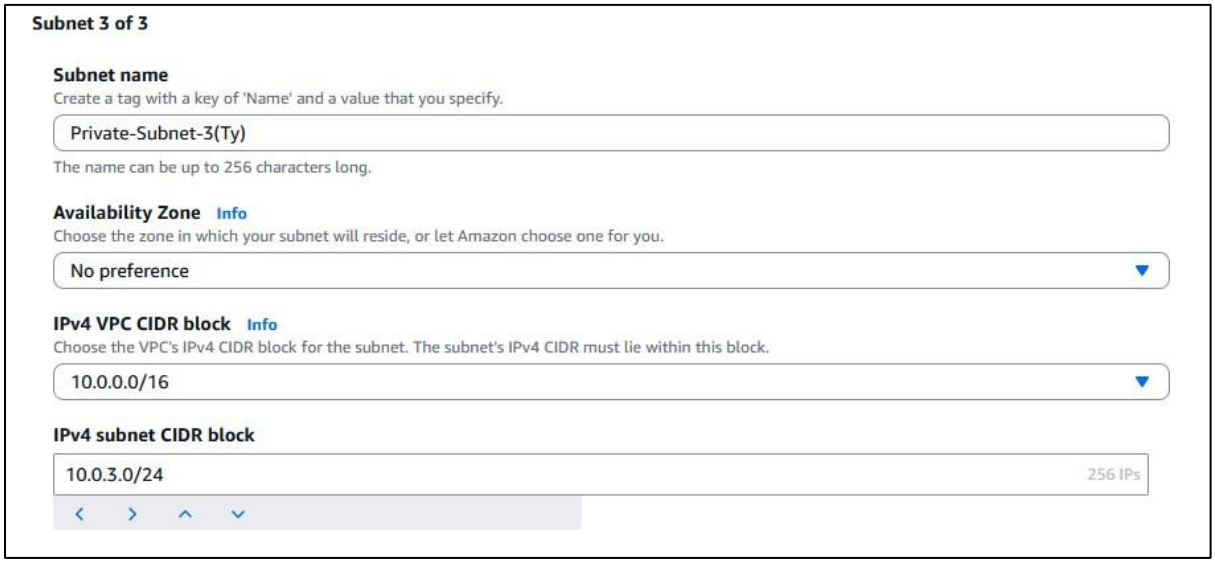
Enable auto-assign public IP for public subnet of VPC B.



Subnet 2



Subnet 3



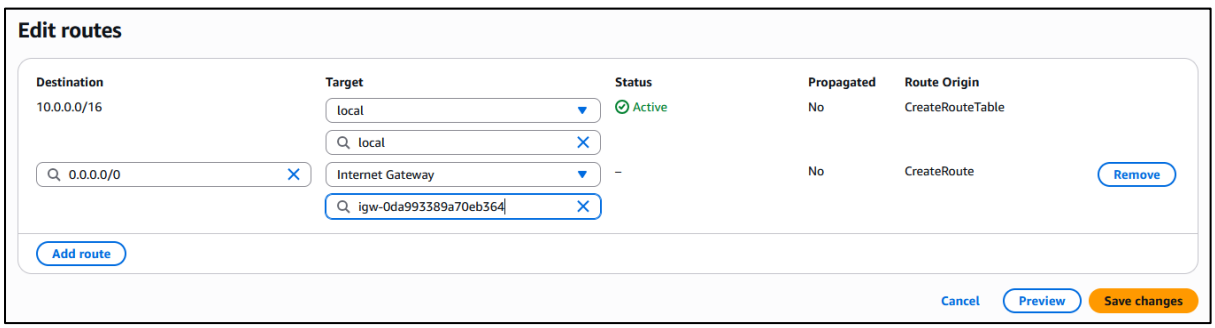
Create Route Tables for all the subnets.

Public Route Table

1. Create **Route Table → Public-RT-Tokyo**
2. **VPC:** VPC-B-Tokyo



3. Go to **Routes** tab:
    - Add route:
        - **Destination:** 0.0.0.0/0
        - **Target:** Internet Gateway (IGW-Tokyo)

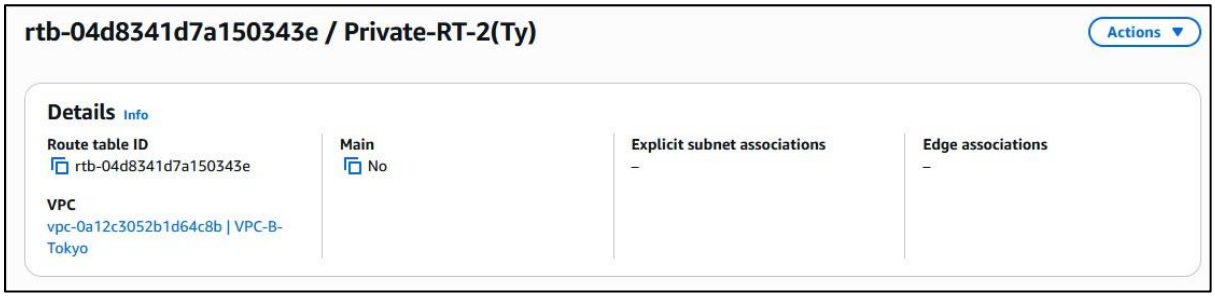


4. Associate with **Public-Subnet-1(Ty)**



Private Route Table for Subnet 2

1. Create **Route Table → Private-RT-2(Ty)**
2. **VPC:** VPC-B-Tokyo

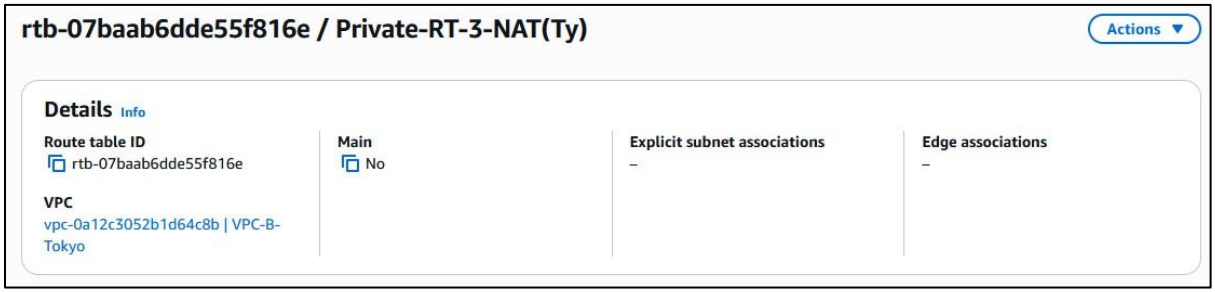


3. Associate with **Private-Subnet-2(Ty)**

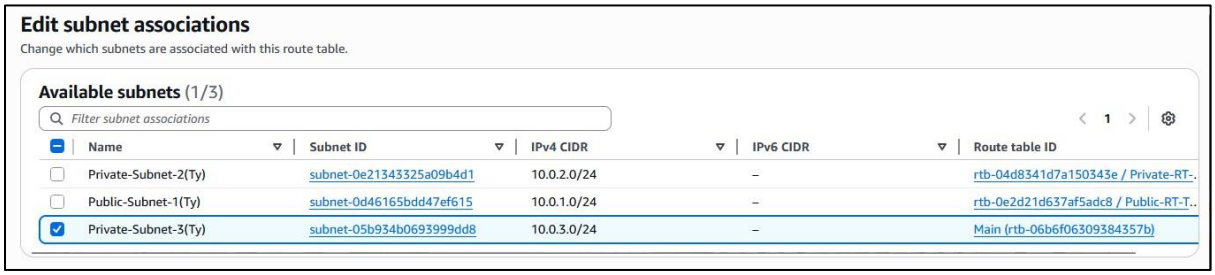


Private Route Table for Subnet 3

1. Create **Route Table → Private-RT-3-NAT(Ty)**
2. **VPC:** VPC-B-Tokyo



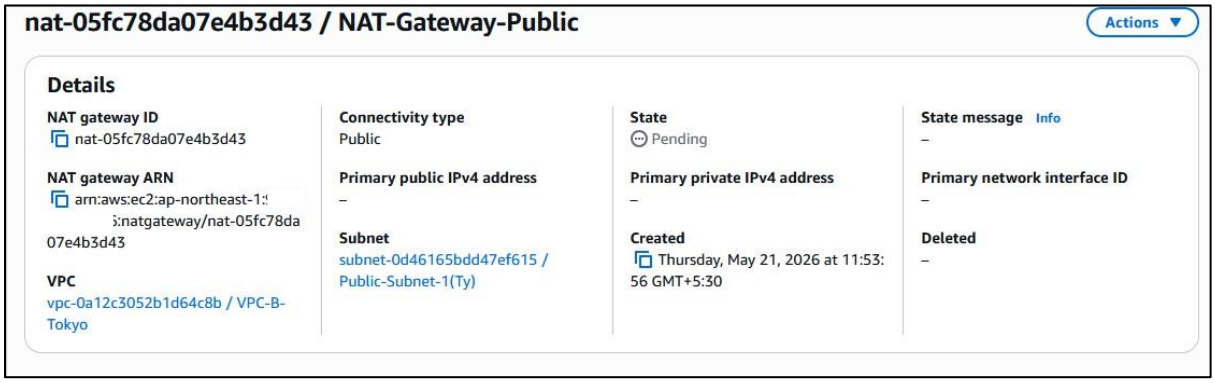
3. Add route:
    - **Destination:** 0.0.0.0/0
    - **Target:** NAT Gateway (refer to Step 3)
4. Associate with **Private-Subnet-3(Ty)**



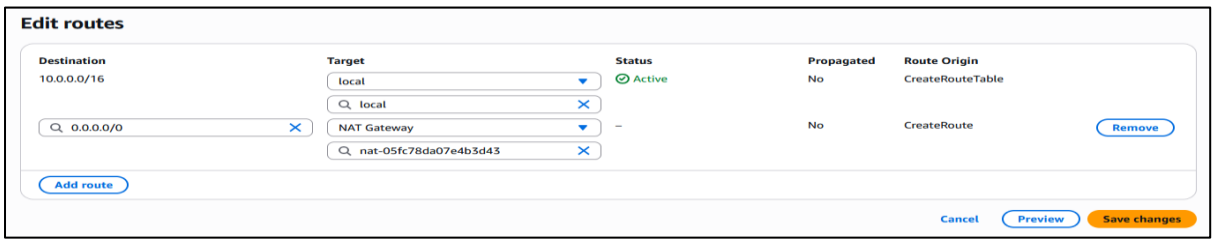
Step 3: Configuring NAT Gateway for Private Subnet Internet Access

NAT Gateway allows Subnet 3 to initiate outbound internet connections while blocking inbound.

1. Go to **NAT Gateways → Create NAT Gateway**
2. Configure:
    - **Subnet:** Public-Subnet-1(Ty)
    - **Elastic IP allocation:** Click **Allocate Elastic IP**
    - **Name:** NAT-Gateway-Public
3. Wait for status to change to ‘Available’



4. Edit **Private-RT-3-NAT(Ty)** route table:
    - Add route
        - **Destination:** 0.0.0.0/0
        - **Target:** NAT Gateway



Step 4: Security Groups Configuration

VPC-A Mumbai Security Group (Mumbai Region)

1. Go to **EC2 → Security Groups → Create Security Group**
2. **Name:** SG-VPC-A-Private
3. **VPC:** VPC-A-Mumbai
4. **Inbound Rules:**
    - Type: SSH, Port: 22, Source: Anywhere IPv4
    - Type: ALL ICMP – IPv4, Source: Anywhere IPv4
5. **Outbound Rules:** default



VPC-B Tokyo – Public Subnet Security Group (Tokyo Region)

1. **Name:** SG-VPC-B-Public
2. **VPC:** VPC-B-Tokyo
3. **Inbound Rules:**
    - Type: SSH, Port: 22, Source: Anywhere IPv4
    - Type: HTTP,Port: 80, Source: Anywhere IPv4
    - Type: HTTPS, Port: 443, Source: Anywhere IPv4
4. **Outbound Rules:** default



VPC-B Tokyo – Private Subnets Security Group (Tokyo Region)

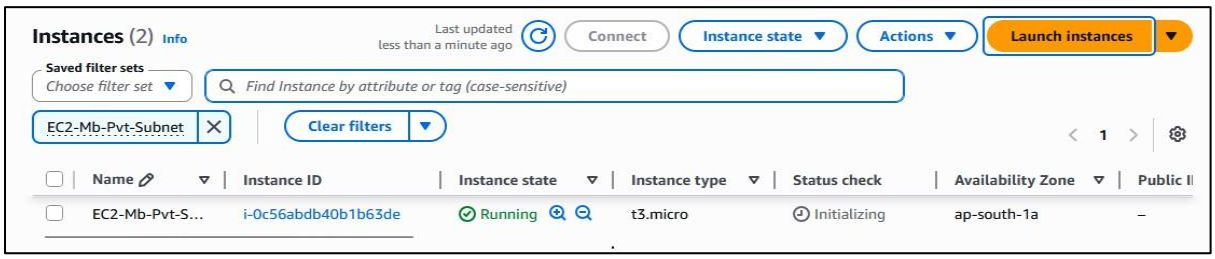
1. **Name:** SG-VPC-B-Private
2. **VPC:** VPC-B-Tokyo
3. **Inbound Rules:**
    - Type: SSH, Port: 22, Source: 0.0.0.0/0
    - Type: ALL ICMP – IPv4, Protocol: ICMP, Source: 0.0.0.0/0
4. **Outbound Rules:** default



Step 5: Launching EC2 Instances

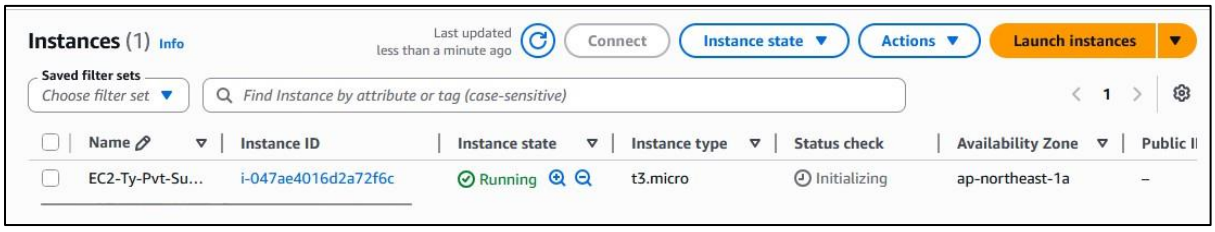
Instance 1: VPC-A Mumbai Private Subnet (Mumbai Region)

1. Go to **EC2 → Instances → Launch Instances**
2. **Name:** EC2-Mb-Pvt-Subnet
3. **AMI Selection:** Amazon Linux 2023 (kernel-6.1)
4. **Instance Details:**
    - **Instance Type:** t3.micro
    - **VPC:** VPC-A-Mumbai
    - **Subnet:** Private-Subnet-1(Mb)
    - **Auto-assign Public IP:**
5. **Security Group:** SG-VPC-A-Private
6. **Key Pair:** Select or create your key pair
7. Review and **Launch Instance**



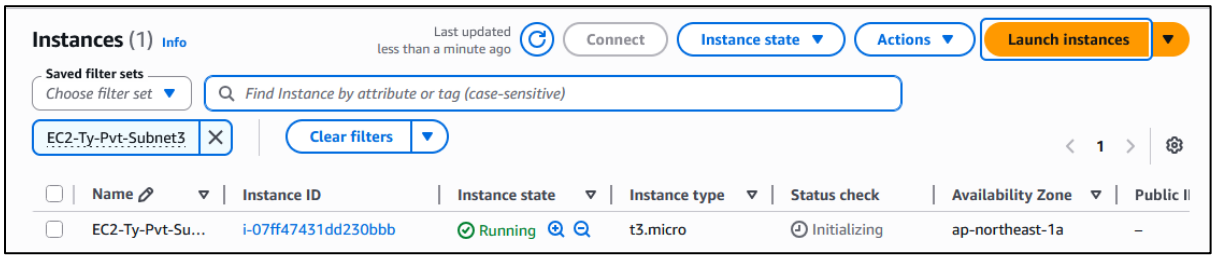
Instance 2: VPC-B Tokyo Private Subnet 2 (Tokyo Region)

1. **Name:** EC2-Ty-Pvt-Subnet2
2. **VPC:** VPC-B-Tokyo
3. **Subnet:** Private-Subnet-2(Ty)
4. **Security Group:** SG-VPC-B-Private



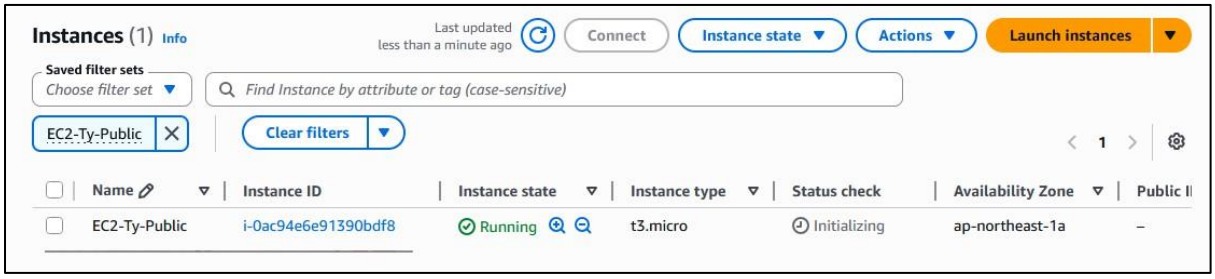
Instance 3: VPC-B Tokyo Private Subnet 3 (Tokyo Region)

1. **Name:** EC2-Ty-Pvt-Subnet3
2. **VPC:** VPC-B-Tokyo
3. **Subnet:** Private-Subnet-3(Ty)
4. **Security Group:** SG-VPC-B-Private



Instance 4: VPC-B Tokyo Public Subnet (Tokyo Reg ion)

1. **Name:** EC2-Ty-Public
2. **VPC:** VPC-B-Tokyo
3. **Subnet:** Public-Subnet-1
4. **Security Group:** SG-VPC-B-Public



Step 6: Establishing Cross-Region VPC Peering

Switch to **Mumbai** region

1. Navigate to **VPC Dashboard → Peering Connections**
2. Configure the following:
    - **Name:** Mumbai-Tokyo-Peering
    - **Requester VPC:** VPC-A-Mumbai
    - **Account:** My Account
    - **Region:** ap-northeast-1 (Tokyo)
    - **Accepter VPC:** VPC-B-Tokyo

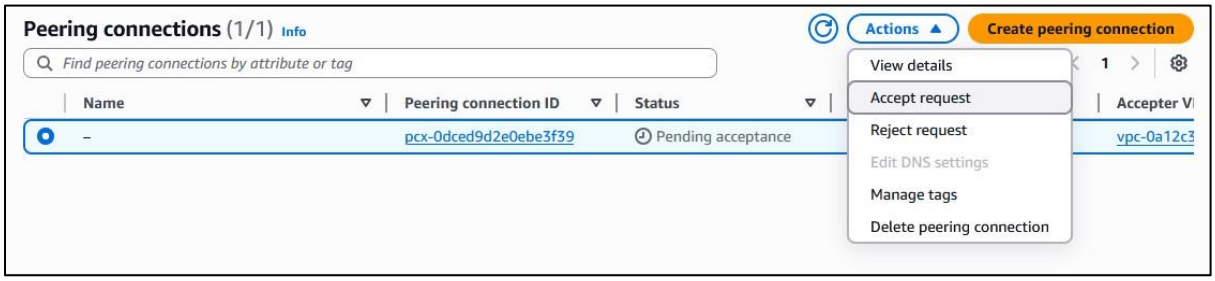
3. Click **Create Peering Connection**



The peering connection status initially appears as **Pending Acceptance**.

Since this was a cross-region peering setup, the request had to be accepted from the Tokyo region. Switch to **Tokyo** region.

1. Navigate to **VPC Dashboard → Peering Connections**
2. Select **Mumbai-Tokyo-Peering**
3. Select **Action → Accept Request**

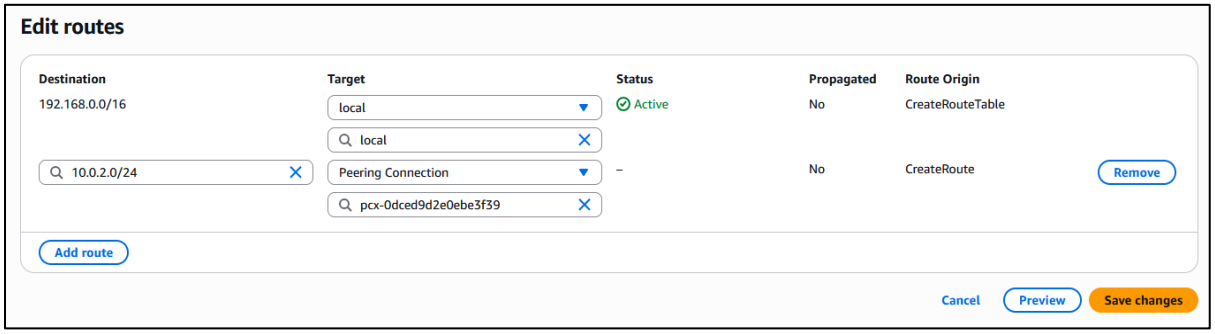


Once accpeted, the peering status chages to **Active.**

The communication would still not work until the toute tables are updated.

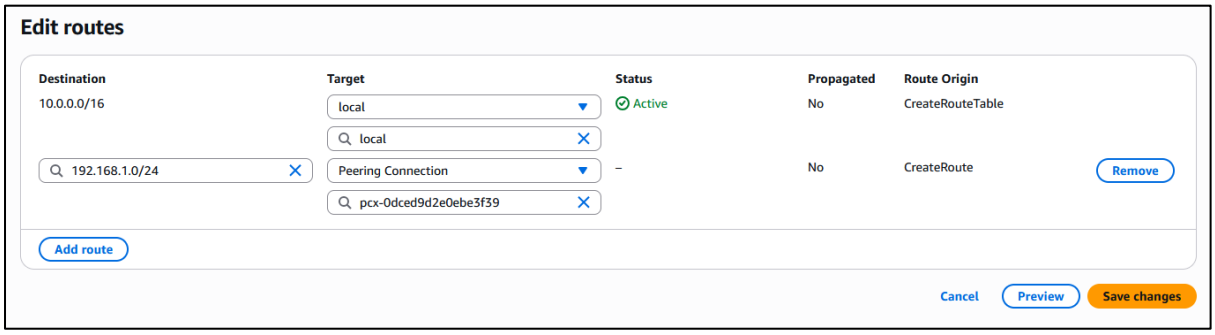
In **Mumbai** region,

1. Go to **Private-RT-1(Mb)** route table (VPC-A)
2. Add route:
    - **Destination:** 10.0.2.0/24 (Private Subnet 2 in Tokyo)
    - **Target:** Peering Connection



In **Tokyo** region,

3. Go to **Private-RT-2(Ty)** route table (VPC-A)
4. Add route:
    - **Destination:** 192.168.1.0/24 (Private Subnet 1 in Mumbai)
    - **Target:** Peering Connection

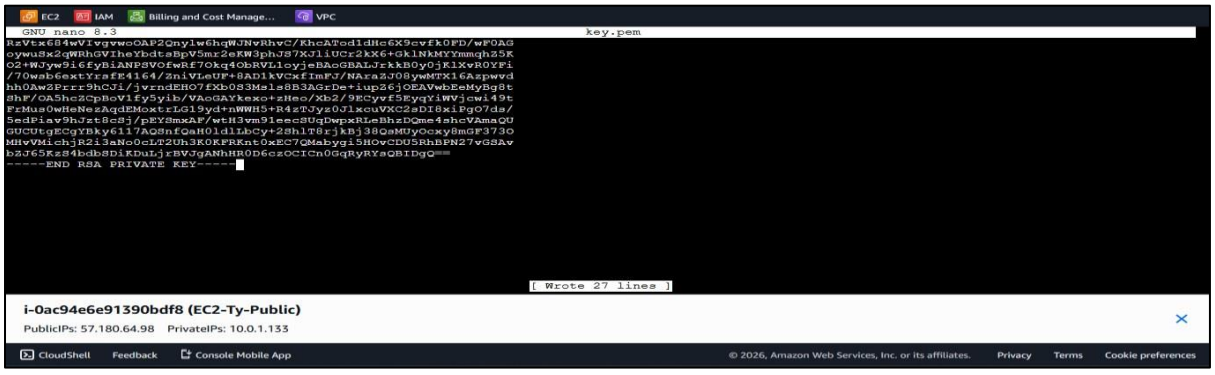


Step 7: Testing Peering Connection

Since the Mumbai EC2 instance resided inside a private subnet without public internet access, the Tokyo public EC2 instance was used as a bastion host to access the private environments securely.

1. Connect to **EC2-Ty-Public**
2. Create a file key.pem using the following command
   $ nano key.pem

   Copy the contents of Key pair of Tokyo to this file and save.



3. The key pair file permissions were updated
   $ chmod 400 key.pem
4. Connect to the private instance of Tokyo region using its private instance IP:
   $ ssh -i key.pem ec2-user@10.0.2.8

```
[root@ip-10-0-1-133 ec2-user]# ssh -i key.pem ec2-user@10.0.2.8
The authenticity of host '10.0.2.8 (10.0.2.8)' can't be established.
ED25519 key fingerprint is SHA256:IzEodH9hw+rsGkGSjkRDfclDP5Ir+fx3ybFUyYu0dB4.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '10.0.2.8' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-2-8 ~]$
```

5. Connect to the private instance of Mumbai region using its private instance IP
   $ ssh -I key.pem ec2-user@192.168.2.223

```
[ec2-user@ip-10-0-2-8 ~]$ ssh -i key.pem ec2-user@192.168.1.223
Warning: Identity file key.pem not accessible: No such file or directory.
The authenticity of host '192.168.1.223 (192.168.1.223)' can't be established.
ED25519 key fingerprint is SHA256:I6uf3zSwzapudAJZXf0YRRYsIm0tqWYdMvl0SmZVZms.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.1.223' (ED25519) to the list of known hosts.
ec2-user@192.168.1.223: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

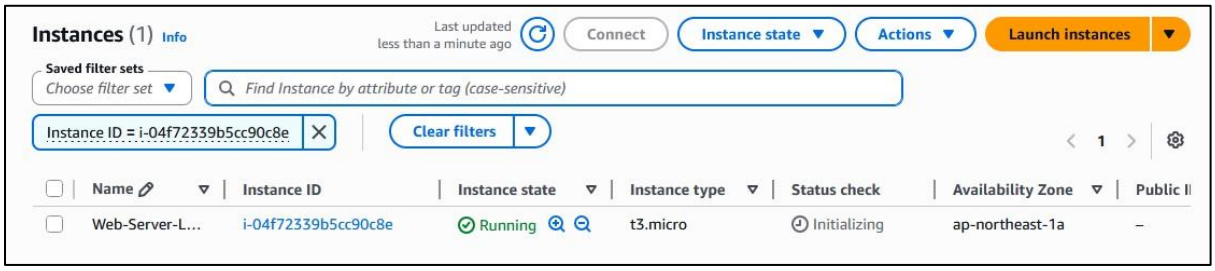
6. Once connected, test communication.

```
[ec2-user@ip-10-0-2-8 ~]$ ping 192.168.1.223
PING 192.168.1.223 (192.168.1.223) 56(84) bytes of data.
64 bytes from 192.168.1.223: icmp_seq=1 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=2 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=3 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=4 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=5 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=6 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=7 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=8 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=9 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=10 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=11 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=12 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=13 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=14 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=15 ttl=127 time=127 ms
```

Step 8: Deploying the LAMP Stack Server

1. Launch a new instance in VPC B in Subnet 1 with the following configurations:
   - **Name:** Web-Server-LAMP

- **AMI:** Ubuntu Server 26.04 LTS
- **Instance Type:** t3.micro
- **VPC:** VPC-B-Tokyo
- **Subnet:** Public-Subnet-1(Ty)
- **Auto Assign Public IP:** Enabled
- **Security Group:** SG-VPC-B-Public



2. After launching the instance, connect to the server and complete the LAMP stack installation and configuration.

   For the detailed step-by-step setup process, including Apache, MySQL, PHP installation, and database connectivity configuration, refer to the following guide: [LAMP Stack Installation Guide]

3. After completing the setup, verify the deployment by accessing the following outputs from the browser:

   https://Public_IP//database_test.php



Conclusion

This project provided a hands-on understanding of how AWS networking components work together in a real-world cloud environment. From configuring multi-region VPCs and secure subnet communication to implementing NAT Gateway routing, VPC peering, bastion host access, and deploying a LAMP stack server, the setup demonstrated the practical side of building secure and scalable cloud infrastructure on AWS.